

MANAGEMENT BRIEFING

FinOps: Visibility & Governance

Seeing cloud spend — and controlling it

Presented to
Executive Leadership

Date

Spend you can see



THE PROBLEM WE FACE

Cloud spend is dynamic, distributed and opaque



Costs move daily

Spend is dynamic and distributed across teams, services and regions.



Budgeting models don't fit

Traditional IT budgeting assumes fixed, predictable costs. Cloud is neither.



Engineers spend money

Technical decisions made daily carry direct, immediate financial consequences.



Bills arrive without context

Monthly invoices show what was spent — rarely who spent it, or why.

“When costs rise and no one can explain who owns the change or what caused it, visibility has failed — even if a dashboard is showing numbers.”

The result

Sticker shock, finger-pointing and reactive cost-cutting instead of strategic optimization.

WHY VISIBILITY FIRST

You cannot optimize what you cannot see

Four questions the business must be able to answer — and the four capabilities that answer them.

Which teams and products drive the spend?



Cost allocation

By team, product and environment — which depends on consistent tagging.

Is spend growing with the business, or faster?



Unit economics

Cost per customer, per transaction, per model call.

What are we paying for and not using?



Granular reporting

By service and resource, over time — where trends and spikes show.

Can we act before the month closes?



Fresh data

Daily, not monthly. Monthly reporting cannot catch a regression.

The goal: move from “IT is overspending” to “Team X used Y, at cost Z, delivering A.”

THE VISIBILITY BLOCKERS

Five filters between us and a clear view



Inconsistent tagging

A significant portion of spend stays unallocated or misallocated.



Shared resources

Databases, Kubernetes clusters and networking resist clean attribution to one team.



Commitments & discounts

Reserved Instances and Savings Plans break attribution — list price $\neq$ price paid.



Multi-cloud estates

Each provider bills and reports differently; a unified view has to be constructed.



SaaS & marketplace spend

Invisible in cloud billing. Teams forget the subscriptions they signed.

“Different clouds report data differently, network traffic adds up fast, and Kubernetes or SaaS often hide who generated the spend.”

The foundation of all visibility



The challenge

“Tagging is the single most foundational habit for sustainable FinOps maturity. It is also the most frequently skipped.”



Define a global tag taxonomy

One master list, agreed across the business — aligned to business need, not just cloud ops.



Enforce tags at creation

Azure Policy, AWS Organizations tag policies, or Terraform. Retrofitting later is painful.



Start with showback

Show teams their spend, uncharged, for 4-6 weeks — gaps surface before money is on the line.



Monitor tag compliance

Dashboards that surface missing or incorrect metadata, tracked as a KPI.

Minimum viable tags

Environment

CostCenter

ApplicationID

Owner

Project

Two costs that resist a simple owner

Tagging solves most allocation. These two need a decision instead.



Shared platforms

Databases, clusters and gateways serve many teams under one set of tags.

- Split on measured consumption, not headcount — it survives the argument
- Freeze the method for the fiscal year and write it down
- Per-team attribution needs usage instrumentation, not tags



Commitments & discounts

Reserved capacity and savings plans mean the rate shown is not the rate paid.

- Report coverage and utilization separately — they fail differently
- Show list and discounted price, and attribute the saving to who earned it
- An unused commitment is prepaid waste, invisible on a list-price report

Start with showback either way. Report each team's share for four to six weeks with nothing charged. It surfaces the allocation disputes while the stakes are low — then chargeback becomes a date rather than an argument.

WHAT A PROJECT ACTUALLY COSTS

The model is one line on a longer bill

An AI feature is quoted at its token cost. It runs on a stack that bills every month regardless.

One project, billed in parts

- Model / token spend** the line everyone quotes
- Compute** app services, containers, nodes
- Databases** SQL, Cosmos, managed Postgres
- Cache & queues** Redis, Service Bus, event hubs
- Storage & backup** documents, embeddings, retention
- Network & gateway** egress, private endpoints, APIM
- Observability** log and metric ingestion, retention

What gets left out of the estimate

- Non-production copies that never switch off
- Log and trace ingestion, which scales with traffic
- Egress and cross-region calls between the parts
- Retention — of documents, embeddings and backups

Report the project, not the resource

One ApplicationID across every resource the project touches, so the review sees a single number and a single owner. Then divide it by the unit that matters — cost per customer, per order, per conversation — and the model share becomes a ratio anyone can argue with.

A project quoted at its model cost alone will be wrong by the size of everything under it — and the surprise arrives at the first full month, not at the pilot.

AI is repeating the cloud journey, at speed

The same three failures — no owner, no attribution, no ceiling — arriving on a much shorter clock.

72%

hit an unexpected AI bill
in the past year

One in three, more than
once.

52%

have no clear owner for
AI cost

Split across engineering,
FinOps, finance and IT.

4 in 5

need a day or more to
trace a spike to its
source

Only about one in five
manage it within hours.

26%

of AI spend estimated
wasted

Respondents' own estimate,
not a measurement.

The number to manage

Time to attribute a spike. It turns an argument about governance into a number we can move — days today, hours as the target — and it is the one metric a runaway AI workload is measured against.

Why a rollup will not do it

AI waste hides in prompt design, model choice and retry logic. A monthly bill by resource cannot name the caller — that needs per-consumer token attribution.

SHIFT LEFT

A seat at the design table

Most of a workload's lifetime cost is settled once the architecture is agreed. FinOps belongs in that conversation, not in the postmortem.

Where FinOps belongs

Requirements

Design

Build

Deploy

Run

Invoice

Where FinOps usually arrives

The exchange in the room

FinOps brings

two or three priced options, a cost target, the allocation plan

FinOps leaves with

the real SLO, what is temporary, which unit carries the value

Six questions before the build starts

- Peak versus average — how spiky is the load?
- How much data, kept how long, leaving the region?
- What does the SLO genuinely require?
- What is temporary, and when does it switch off?
- For AI: tokens, cache rate, model tier
- Which owner, and which unit do we divide by?

The output is a cost target recorded beside the latency and availability targets. It works where FinOps sits with engineers rather than auditing them.

OWNERSHIP

Every workload has a named owner

A person, not a distribution list. A group address answers nothing at 2am on a Sunday.



Owns the budget

Signs it off and explains the variance.



Answers the alerts

The routing target for their workload.



Reviews monthly

With their own numbers, not a summary.



Decommissions

Nothing else retires unused resources.

And who owns the practice

Finance	budgets and allocation policy
Platform / FinOps	the standard, the data, the tooling
Engineering	guardrails, instrumentation, response
Leadership	commitments and the trade-offs

Unowned is not unbudgeted

A resource with no owner is charged to the central budget — the one place cost is hardest to challenge. Make ownership a deployment requirement, and give orphans a standing rule: no owner, no renewal.

FinOps fails when it is one team's job — no single function above owns more than one line of it.

THE CONTROLS

A report is not a control

Three mechanisms, ordered by how fast each one acts on spend that is already running.



Quotas and rate limits

seconds

Token and request caps, gateway throttling, autoscale ceilings. The only control that stops spend in progress — and reversible in minutes, which is what makes it safe to set.



Alerts, tiered by what they prevent

minutes to days

Usage-plane alerts page an owner. Billing-plane alerts go to a channel. Advisory alerts become tickets. Never page on a budget number — and alert on silence too, because a stopped feed looks like a good month.



Budget thresholds

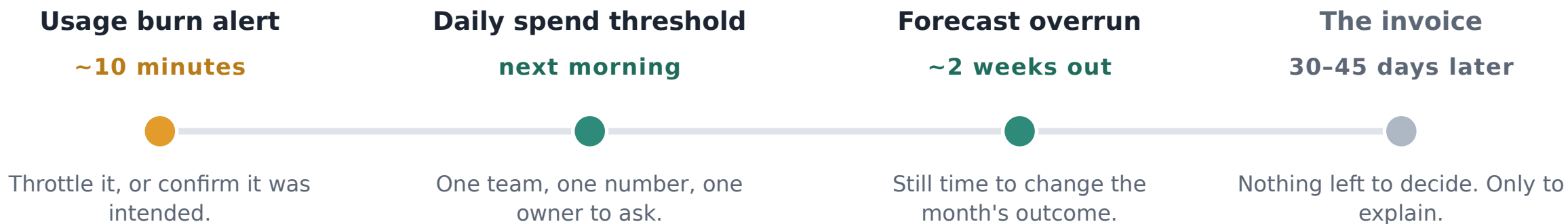
within the month

Notify, review, gate — measured against the elapsed month, so day 10 at 60% of budget is the alert. A flat 80% fires on day 24 of a healthy month and day 8 of a catastrophic one.

“A cost alert cannot save you from a runaway job — by the time the billing data moves, the money is spent.”

Nobody should learn it from the invoice

The same overrun, found at four different moments. Only the first three leave you anything to do about it.



What has to be true

- ✓ Protective alerts run on usage data, not on billing data
- ✓ Every alert has a named owner to route to
- ✓ Budgets are phased, so drift shows up in week one
- ✓ Quotas cap the worst case while someone investigates

The point

The goal is not a smaller invoice. It is an invoice with nothing in it you have not already seen, explained and decided about.

What a FinOps platform adds: Finout

The capabilities a dedicated platform brings over native billing consoles, and what to test before buying one.



One normalized bill

Cloud, SaaS and AI in one view — AWS, Azure, GCP, OCI, Kubernetes, Snowflake, Databricks, OpenAI, Anthropic. FOCUS supported.



Virtual tags

Allocates untagged and shared cost by rule, without touching infrastructure tags — showback without a retagging project.



Kubernetes to pod level

Cluster, namespace and pod allocation without an agent, which is the shared-cost problem most tools leave unsolved.



Dashboards and unit economics

Widgets for cost, usage, waste, budget and unit cost; business metrics from Datadog, Salesforce or Looker to divide by.



Budgets, forecast, anomalies

Budget structures, forecasting on any unit cost, and anomaly alerts on unexpected movement.



Waste detection with an owner

CostGuard consolidates recommendations across providers, assigns an owner and tracks the saving — into Jira, Slack or ServiceNow.

Before buying, prove three things on our own data: that it allocates the spend we cannot allocate today, that it attributes Azure OpenAI cost per calling application rather than per resource, and that it exports FOCUS so we are not locked in. Vendor-published outcomes — around 30% cost reduction and 50% engineer time saved — are marketing claims, not a forecast for us.

Where to start, and what it buys

01

Tagging standard

Agreed and enforced at deployment, with an owner per application.

02

Biggest spend first

Attack the largest categories. Perfect granularity everywhere is a trap.

03

Guardrails on

Quotas and budget thresholds before they are needed, not after.

04

Finance in the room

A monthly review with real numbers, and a date for chargeback.

How we will know it is working

Allocation coverage under 5%, reported weekly

Time to attribute a spike hours, not days

Findings owned and dated adoption, not accuracy

What it buys

Not a smaller invoice — an invoice with nothing in it we have not already seen, explained and decided about. Visibility makes ownership possible, ownership changes behaviour, and changed behaviour is the only saving that survives the quarter.

Two decisions today: who owns the tagging standard, and whether we are willing to have a limit that actually blocks.

Questions & discussion

FinOps: Visibility & Governance — seeing cloud spend, and controlling it

Key contacts

_____	· Title
_____	· Title

Resources

FinOps Foundation	·	finops.org
FOCUS standard	·	focus.finops.org

